

CCNA 200-301 V2.0

21 DNS, and Diagnosing Records

Part IV — Network Services and Security

EXAM TOPICS

4.4

LECTURER

Dr. Mustafa Hameed



THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

Where this chapter sits

Part I	Foundations	chapters 1–7
Part II	Switching and Network Access	chapters 8–14
Part III	IP Routing	chapters 15–19
Part IV	Network Services and Security	chapters 20–26
Part V	Wireless, Virtualisation and Operations	chapters 27–32
Part VI	Sitting the Exam	chapters 33–34

What you will be able to do

- **Trace** a name resolution from a stub resolver to an authoritative server, and say which step a given failure is stuck at.
- **Identify** the six record types the blueprint names — A, AAAA, CNAME, MX, NS and PTR — and state what each is for.
- **Use** `nslookup` and `dig` to query a specific record type against a specific server.
- **Diagnose** the three failure classes the topic names: a host unreachable by name, a web application that half works, and mail that is rejected.

Before we start

Chapter 6 for UDP, TCP and ports. Chapter 20 — the DNS server address usually arrives in a lease, so a DNS fault is sometimes a DHCP option fault. Chapter 5 for what an AAAA record contains.

Vocabulary for this chapter

resolver

recursion

authoritative

TTL

zone

A

AAAA

CNAME

MX

NS

PTR

NXDOMAIN

SERVFAIL

Each is defined where it first matters — not here.

Read the topic wording carefully

This is what the blueprint actually asks for

Topic **4.4** is *diagnose issues with DNS records (A, AAAA, CNAME, MX, NS, and PTR) to support host, web application, and mail server access by name.*

Three things follow. The verb is *diagnose*, so you will be given a resolution that is wrong and asked why. The record types are enumerated, so those six and no others are examinable — there is no SOA, SRV or TXT question waiting for you. And the three purposes are enumerated too, which is a strong hint that questions will be framed as “the website works but mail bounces” rather than as “define a CNAME”.

You are not asked to configure a DNS server. You are asked to read one.

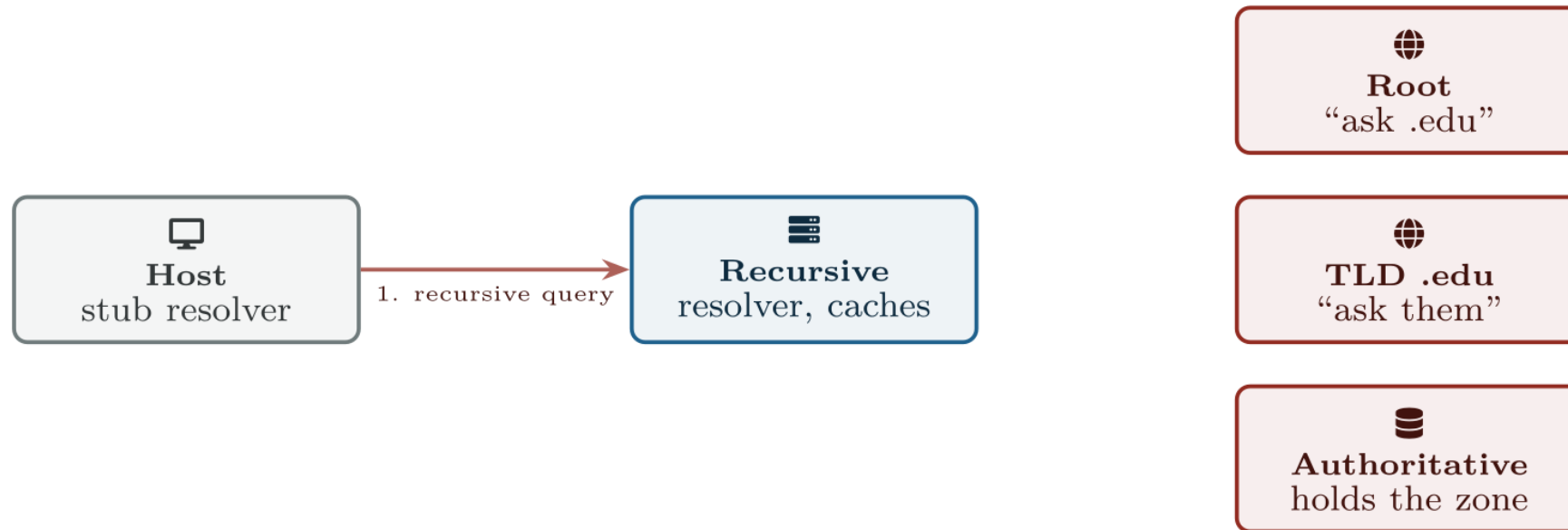
How a name becomes an address



DNS

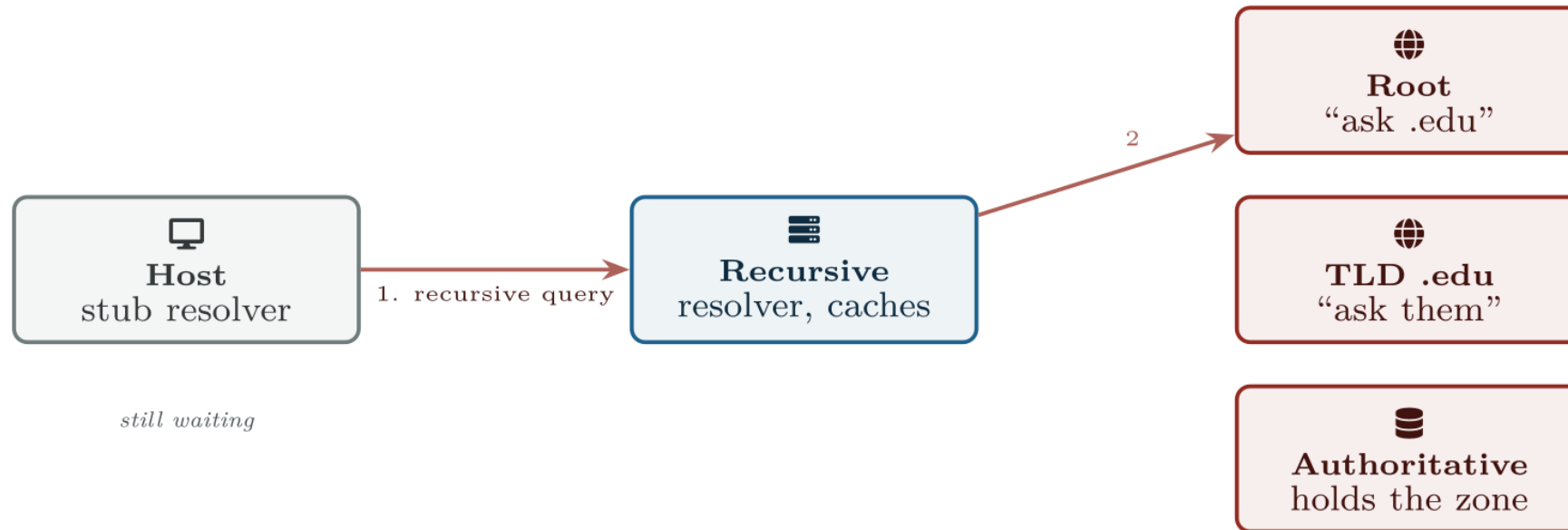
A distributed, hierarchical database mapping names to records. Queries normally travel over UDP port 53; responses too large for a datagram, and zone transfers between servers, use TCP port 53.

How a name becomes an address



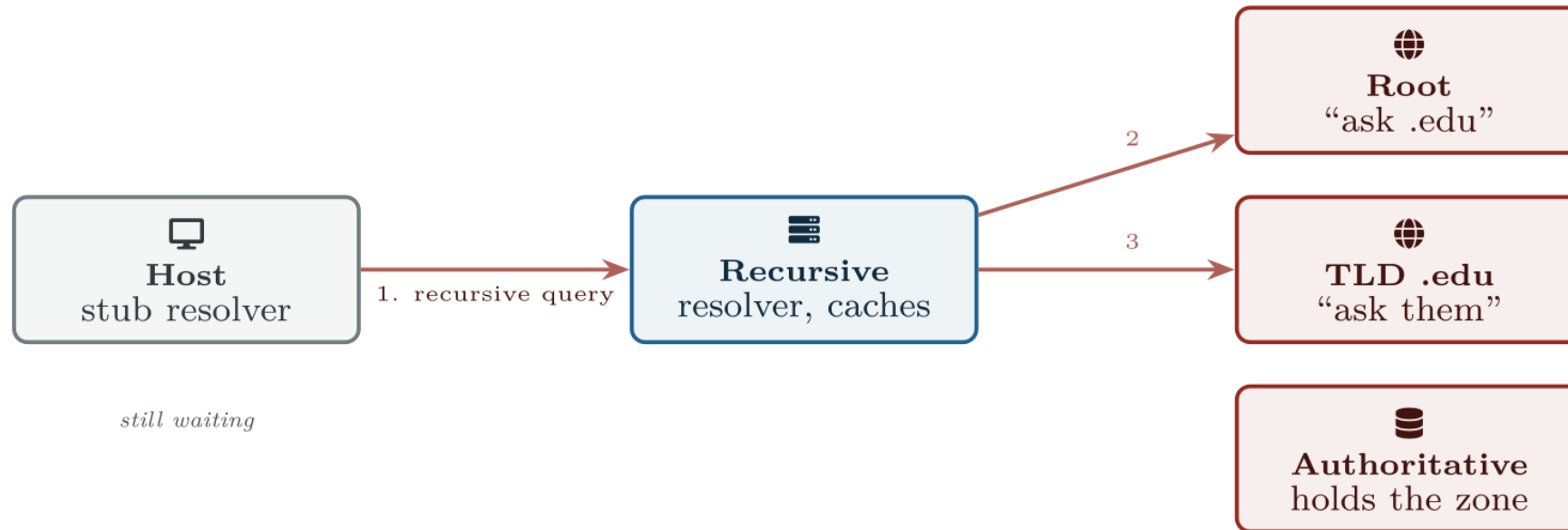
One question, then the host waits. This arrow is the only recursive one in the picture.

How a name becomes an address



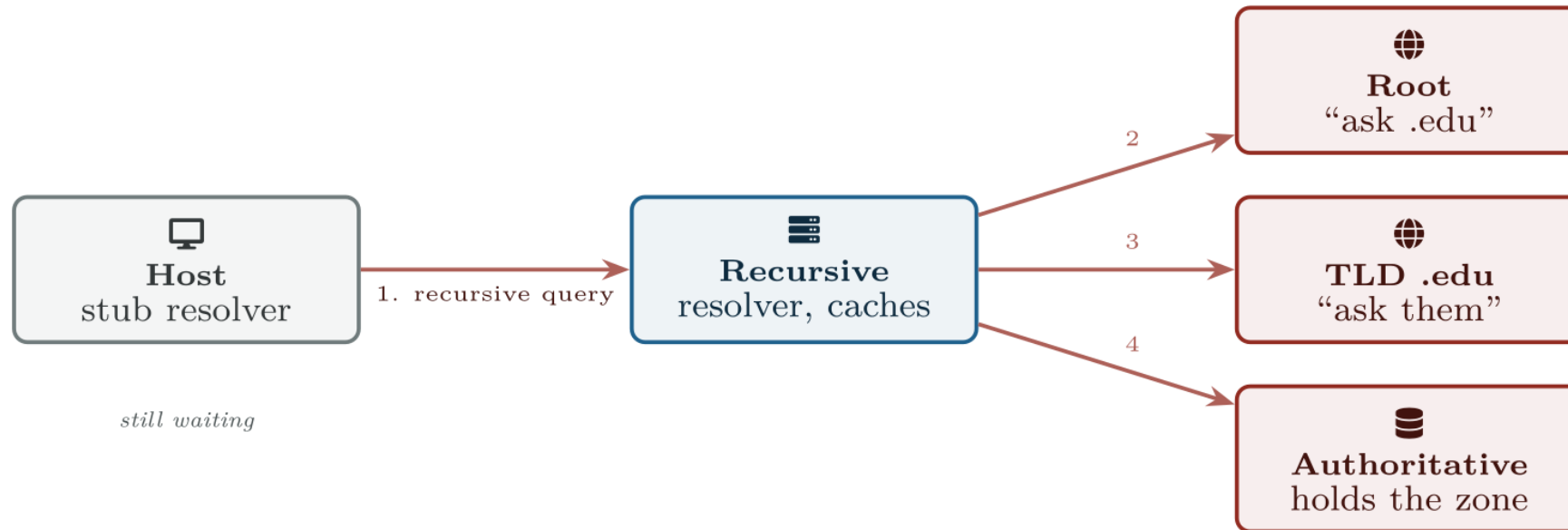
The root does not know the answer — it knows who runs .edu.

How a name becomes an address



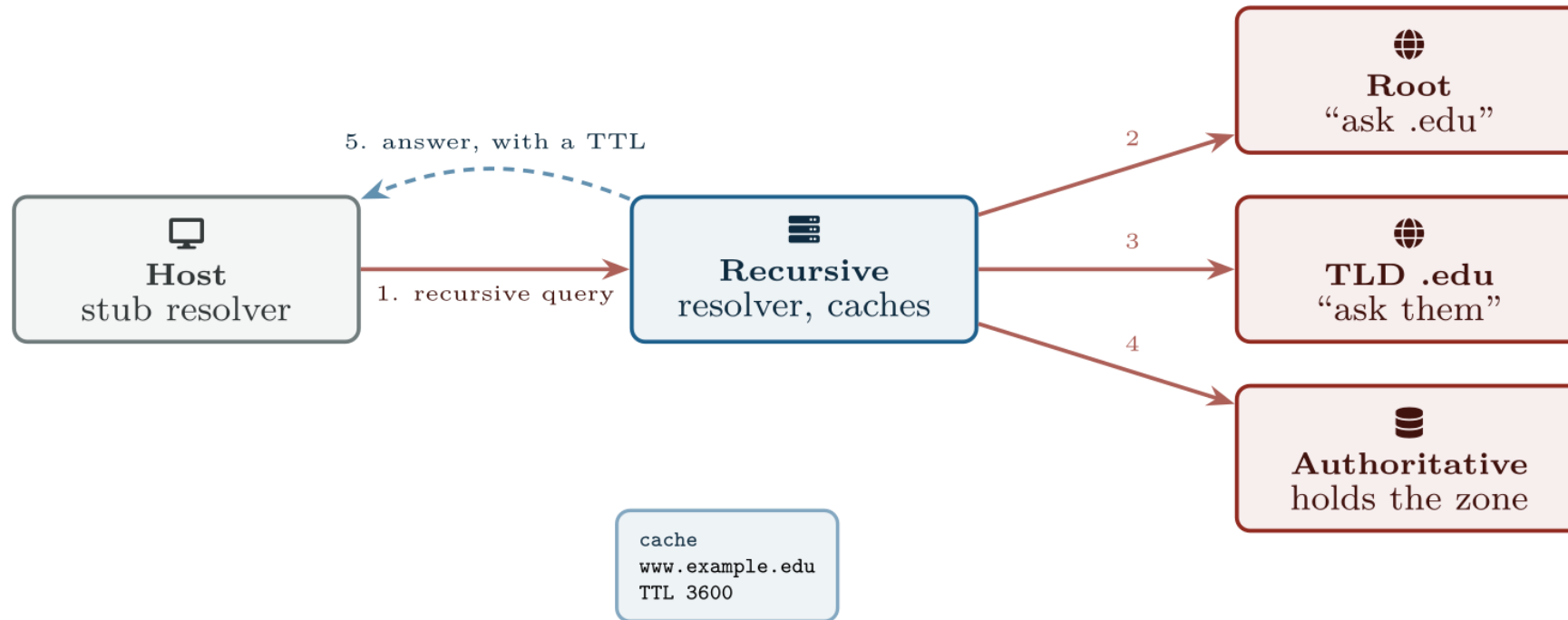
Nor does the TLD. Each referral is one step down the name, right to left.

How a name becomes an address



The authoritative server holds the zone, so this one answers.

How a name becomes an address



The host gets one answer and never learns any of that happened — and the resolver keeps it for the TTL, so the next host skips steps 2 to 4.

The stub resolver explains most “it works for me” arguments

Your host is not doing the work in that figure. It sends one query to whichever resolver DHCP gave it and waits for a complete answer. Everything downstream — which root it used, whether the TLD was slow, whether a cached record is stale — is invisible to the host and different for every resolver.

So when one user cannot reach a site and another can, the first question is not “what is wrong with the site” but **“are they using the same resolver?”**. `ipconfig /all` or `resolvectl status` answers it in one line, and the answer resolves the argument more often than not.

The practical consequence

Lower the TTL *before* a planned change, not after. Drop it to 300 seconds a day ahead, make the change, verify, then raise it again. Doing this in the right order is the difference between a five-minute cutover and a day of intermittent failure — and “why did the change take so long to appear” is a fair short-answer exam question.

Reverse lookups are a separate tree, not a mirror

A PTR record does not live in your forward zone. It lives in `in-addr.arpa` for IPv4 (or `ip6.arpa` for IPv6), with the octets reversed, and it is delegated to *whoever owns the address block* — normally your ISP or cloud provider, not you.

That is why forward and reverse so often disagree. Adding an A record for a new mail server takes you thirty seconds; adding the matching PTR may require a support ticket to somebody else. The exam likes this asymmetry because it explains a real failure: mail that leaves cleanly and is rejected at the far end.

Querying: the two tools



nslookup -type=MX example.edu 8.8.8.8

Windows

Server: dns.google

Address: 8.8.8.8

Non-authoritative answer:

example.edu MX preference = 10, mail exchanger = mail1.example.edu

example.edu MX preference = 20, mail exchanger = mail2.example.edu

mail1.example.edu internet address = 203.0.113.25

mail2.example.edu internet address = 203.0.113.26

dig +noall +answer www.example.edu A

Linux or macOS

```
www.example.edu.      3600    IN      CNAME   web-lb.example.edu.  
web-lb.example.edu.   300     IN      A       203.0.113.10
```

Read the status line first

This is what the blueprint actually asks for

When `dig` is run without `+noall +answer`, the header carries a status, and it narrows the fault immediately:

- **NOERROR** with an empty answer section — the name exists, but not with that record type. Very often the real answer in an exam question.
- **NXDOMAIN** — the name does not exist. A typo, or a record never created.
- **SERVFAIL** — the resolver could not complete. Broken delegation, an unreachable authoritative server, or a validation failure.
- **REFUSED** — the server will not answer you. Usually an access control, not a record problem.

NOERROR-with-no-answer and **NXDOMAIN** look identical to a user and mean completely different things to you.

DNS on IOS itself



Name resolution on a router

IOS · configuration mode

```
ip domain lookup
ip name-server 192.168.20.10 192.168.20.11
ip domain name campus.example.edu
!
```

*! A local override, no server needed. Useful in a lab and useful when
! you want a name to resolve even if DNS is the thing that is broken.*

```
ip host CORE1 10.0.0.1
```

The mistyped-command hang

Type a command wrong in EXEC mode — `shwo ip route` — and IOS assumes you meant to telnet to a host called `shwo`. With `ip domain lookup` enabled and no reachable server, the CLI freezes for tens of seconds on every typo.

On a lab device the fix is `no ip domain lookup`. On a production device, where you may want resolution, use `line con 0` then `transport preferred none`, which stops the CLI from treating unknown words as hostnames without disabling DNS.

Diagnosing the three named failures



MX must point at a name with an A record, never at a CNAME

Two rules, both examinable. The value of an MX record must be a hostname, not an IP address. And that hostname must resolve by A or AAAA directly — pointing an MX at a CNAME is prohibited by the standards and rejected by some mail servers and tolerated by others, which produces the worst kind of fault: mail from *some* senders is delivered and from others is not.

Common pitfalls

- **Querying without `-type=`.** You get A records, see them, and wrongly declare the MX or NS records missing.
- **Not naming a server in the query.** You test your own cache instead of the truth. Always query the authoritative server too.
- **Treating `NXDOMAIN` and `empty-NOERROR` as the same.** One means the name is absent; the other means the *record type* is.
- **Forgetting TTL.** A correct fix looks like a failed fix for as long as the old TTL.
- **Assuming reverse DNS is yours to fix.** PTR usually lives with whoever owns the address block.

Common pitfalls (continued)

- **Blaming DNS for a routing fault.** If `ping` by address also fails, DNS is not involved.

The website works. Mail to the domain bounces with “no route to host”.

`example.edu` loads correctly in a browser from everywhere. Mail sent to `staff@example.edu` from outside is returned by the sender's server. Mail *between* internal users is fine. The mail server itself is up, is listening on port 25, and can be reached by IP address from outside.

The website works. Mail to the domain bounces with “no route to host”.

What would you check first — and what do you expect to see?

dig +noall +answer example.edu MX

Linux#

```
example.edu.      3600    IN      MX      10 mail.example.edu.
```

dig +noall +answer mail.example.edu A

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. The MX record exists and is syntactically fine. The name it points at, `mail.example.edu`, has no A record — the answer section is empty. A sending server looks up the MX, gets a name, tries to resolve that name to an address, gets nothing, and has nowhere to connect to.

Everything else being healthy is what makes this hard to spot. The web works because `www` has its own A record. Internal mail works because internal clients are configured with the server's address directly and never consult the MX. Only *external inbound* mail follows the full MX-then-A path, so only external inbound mail fails.

Why it is doing that

The tell is the shape of the failure: one direction, one protocol, one class of sender. Faults that broad in effect but that narrow in scope are almost always in a record that only one code path reads.

And the lesson

Fix. Add the missing A record for `mail.example.edu` pointing at the server's public address, then re-query to confirm. Two further checks before closing it:

Diagnosing a zone you did not build

Platform note. Packet Tracer's DNS server holds only A and CNAME records and cannot express MX, NS or PTR, so it cannot support this lab. Use a real resolver in CML or GNS3, or run BIND in a container alongside a virtual network.

Topology. One router with `ip name-server` configured, one DNS server holding `example.edu`, and two clients — one dual-stack, one IPv4 only.

1. Load the zone supplied with this lab. Do not read the zone file yet; diagnose from the client side first, as you would in practice.
2. For each of the six record types, run a query that returns it, using `nslookup` on one client and `dig` on the other. Record the exact command in each case.
3. Find every fault in the zone using queries alone. There are four. Write a one-line diagnosis for each *before* opening the zone file.
4. Open the zone file and check your diagnoses. Note any fault you missed and what query would have found it.

Diagnosing a zone you did not build (continued)

5. **Break 1.** Point `www` at a CNAME chain three deep, with the last entry pointing back at the first. Query it and describe the result.
6. **Break 2.** Add an AAAA record for `www` pointing at an unreachable IPv6 address. Browse from the dual-stack client and from the IPv4-only client. Time both. Explain the difference to somebody who has not read this chapter.
7. **Break 3.** Set the A record TTL to 3600, change the address, and observe how long the old value survives on a client that had already queried it. Then repeat with a TTL of 60.
8. **Extension.** Disable `ip domain lookup` on the router, mistype a command, and compare the behaviour with it enabled and no reachable server.

CHECKPOINT 1 of 5

ping www.example.edu fails and ping 203.0.113.10 succeeds. What has this proved, and what is the next command you run?

Discuss · then advance

Checkpoint 1 of 5

ping www.example.edu fails and ping 203.0.113.10 succeeds. What has this proved, and what is the next command you run?

That the network path and the server are fine and the fault is name resolution. Next, query the record explicitly and against a known server: `nslookup -type=A www.example.edu 8.8.8.8`, and compare with what your own resolver returns.

CHECKPOINT 2 of 5

What is the difference between NXDOMAIN and NOERROR with an empty answer section?

Discuss · then advance

Checkpoint 2 of 5

What is the difference between NXDOMAIN and NOERROR with an empty answer section?

NXDOMAIN means the *name* does not exist. **NOERROR** with an empty answer means the name exists but has no record *of the type you asked for*. They look identical to a user and mean entirely different things.

CHECKPOINT 3 of 5

Why can a name that is a CNAME not also hold an MX record?

Discuss · then advance

Checkpoint 3 of 5

Why can a name that is a CNAME not also hold an MX record?

Because a CNAME means “this name is an alias for that name”, and a name that is an alias cannot also carry data of its own. Standards prohibit other records alongside it, and mail servers differ in how they cope — which produces the worst symptom, where mail from some senders arrives and from others does not.

CHECKPOINT 4 of 5

A site is slow for some users and instant for others. Which record type would you check first, and why does it affect only some clients?

Discuss · then advance

Checkpoint 4 of 5

A site is slow for some users and instant for others. Which record type would you check first, and why does it affect only some clients?

AAAA. A dual-stack client prefers IPv6 and will try the AAAA address first; if it is wrong or unreachable the client waits for a timeout before falling back to IPv4. IPv4-only clients never see it, so the site is slow for some and instant for others.

CHECKPOINT 5 of 5

Whose responsibility is a PTR record for your public address, and what breaks when it is missing?

Discuss · then advance

Checkpoint 5 of 5

Whose responsibility is a PTR record for your public address, and what breaks when it is missing?

Usually whoever owns the address block — your ISP or cloud provider, not you, because reverse DNS is delegated separately in `in-addr.arpa`. Without it, outbound mail is commonly rejected or scored as spam, because receiving servers check that the connecting address reverse-resolves and forward-resolves back.

What to take away

- A host runs a stub resolver: one query, one answer. All the recursion happens at the resolver, which caches for the TTL.
- Six examinable types: A and AAAA give addresses, CNAME gives another name, MX gives a mail exchanger with a preference, NS delegates a zone, PTR reverses an address into a name.
- Query a *type* and a *server* explicitly, or you are testing neither the record you meant nor the source you meant.
- **NXDOMAIN** is a missing name; empty **NOERROR** is a missing record type; **SERVFAIL** is usually a broken delegation.

What to take away (continued)

- Host fault: does it ping by address? Web fault: suspect AAAA or an apex CNAME. Mail fault: MX inbound, PTR outbound, NS if everything is gone.
- MX must name a host that has an A record, and must not name a CNAME.
- TTL means a correct fix takes time to become visible. Lower it before a change, not after.

Where we got to

- A host runs a stub resolver: one query, one answer. All the recursion happens at the resolver, which caches for the TTL.
- Six examinable types: A and AAAA give addresses, CNAME gives another name, MX gives a mail exchanger with a preference, NS delegates a zone, PTR reverses an address into a name.
- Query a *type* and a *server* explicitly, or you are testing neither the record you meant nor the source you meant.
- NXDOMAIN is a missing name; empty NOERROR is a missing record type; SERVFAIL is usually a broken delegation.

NEXT

Chapter 22 — NAT and PAT on IOS XE